

Introduction and the Verification Frame

ECON 895 Spatial Techniques in Empirical Economics

Noel D. Johnson
George Mason University

Week 1 August 24, 2026

njohnsoL@gmu.edu

Plan for tonight

What we do here

Why this course still teaches code

The one idea you need tonight

Verification, performed

How the course runs

Your paper

Before next week

We break around 8:45.

What we do here

What this course is

- An introduction to using **spatial data** and **text data** in applied economic research.
- You will learn to create and analyze spatial data in R, to digitize archival maps in QGIS, and to build and validate measures from text.
- We read papers from Urban, Growth, Trade, Economic History, and Development.

This is a course about producing research, not about completing assignments. Every tool we cover exists to get you closer to a publishable paper.

This is what spatial data looks like

African cities and national borders



51 country polygons and 2,541 city points. Two files, one map, drawn by four lines of R.

Why economists went spatial

- Satellite and remotely sensed data opened outcomes we could never measure before, at fine resolution, everywhere on earth (Donaldson and Storeygard, 2016).
- Spatial variation lets us study development *below* the country level, where institutions, geography, and history actually vary (Michalopoulos and Papaioannou, 2018).
- History scattered its treatments across space. Boundaries, roads, rivers, and shocks become research designs.

The unit of observation stopped being the country. That is the whole story, and it is why a course like this exists at all.

The papers this buys you

Four you will read this semester

- Did the printing press cause city growth? **Distance from Mainz** as an instrument (**Dittmar, 2011**). Week 2.
- Do colonial institutions still matter centuries later? A **boundary** as a regression discontinuity (**Dell, 2010**). Week 3.
- What did the potato do to Old World population? **Suitability** crossed with timing, a difference in differences (**Nunn and Qian, 2011**). Week 4.
- What were the railroads worth? **Market access** computed from the whole network (**Donaldson and Hornbeck, 2016**). Week 7.

Each week pairs one spatial technique with one famous paper that runs on it. The technique is not decoration. It *is* the identification strategy.

And text, now, too

- Text is data. Newspapers, parliamentary records, book titles, court records, firm filings ([Gentzkow et al., 2019](#)).
- The pipeline from raw text to a number in a regression involves choices, and the choices change results ([Denny and Spirling, 2018](#)).
- LLMs turned text measurement from a specialist skill into something every applied economist will touch. That makes validation everybody's problem.

Weeks 10 to 12. Concepts first, so that when you point a model at a corpus you know what you asked for and how to check what came back.

Why this course still teaches code

The obvious question

- AI tools now write most first draft code.
- A model will produce working `sf` code from a one sentence prompt.
- So why spend fourteen weeks learning to write it yourself?

The question deserves a serious answer, because the honest version of this course in 2019 no longer exists. Teaching you to type `st_buffer` is not worth your tuition. *Teaching you to know when `st_buffer` silently did the wrong thing is.*

The false choice

One view

- The tools write code now.
- Learning to code is obsolete.
- Teach prompting instead.

The other view

- Ban the tools.
- Code by hand forever.
- Pretend it is 2019.

Both views fail the same way

They treat code and AI as substitutes. In research practice they are complements, and the thing that links them is **verification**.

The verification frame

The fundamentals matter more with LLMs, not less

1. Verifying what the model did requires the building blocks.
 2. Directing the model well requires knowing exactly what to ask for.
 3. The models are very good at finding discrepancies between your code and your paper, once you know to ask.
- A student who cannot read `sf` code cannot check what the model wrote.
 - The course teaches code precisely so you can direct and audit the machine.

You sign the result

- This is the research norm behind the whole course.
- The author is the residual claimant of the paper. The author is also its monitor.
- No referee, no coauthor, and no model is responsible for your numbers. You are.

The signing rule

Every number and every line of code in work carrying your name is yours. Submitting it asserts that you understand it and have checked it. This holds whether you typed the code or a model did.

So far this is a speech

Everything I have said tonight is easy to say and costs me nothing. Any course could put “verification” on a slide.

- So for the next half hour we are going to do it instead.
- I will write code in front of you. Some of it will be wrong.
- **Your job is to tell me, before I run each line, what you think will happen.** You will be wrong sometimes too. That is the exercise.

If you leave tonight having watched a plausible, error-free, completely wrong number appear on this screen, the evening did its work.

The one idea you need tonight

The earth is round. Your screen is not.

- Every map you have ever seen is a lie of a specific, documented kind.
- You cannot flatten a sphere without tearing or stretching it. That is a theorem, not an engineering limitation.
- So every projection chooses what to preserve, and therefore what to sacrifice. Area, shape, distance, direction. Pick at most some.

Coordinate reference system, or CRS

The rule that maps a location on the earth to a pair of numbers. It says what the numbers mean, what units they are in, and what shape of earth they assume.

Perhaps someone should brief the President

Mercator (EPSG:3857)



Mollweide (equal area)



Greenland and **the DRC**, same two countries, two projections.

Put a number on it

##	<i>true km2</i>	<i>Mercator km2</i>	<i>inflated</i>
## <i>Greenland</i>	2,194,714	36,285,503	16.5x
## <i>Democratic R</i>	2,338,772	2,358,240	1.0x

- The DRC is larger than Greenland in reality. On a Mercator map Greenland looks several times larger.
- **This is not a rendering quirk.** If you compute area in EPSG:3857, those inflated numbers are what your regression gets.

Two families, and the only distinction that matters tonight

Geographic

- Longitude and latitude.
- Units are **degrees**.
- EPSG:4326 is the common one, the GPS default.
- Angles on a sphere, not positions on a plane.

Projected

- Flattened onto a plane.
- Units are **linear**. Metres, or feet.
- EPSG:5070, 32632, 2283, thousands more.
- Each one distorts something on purpose.

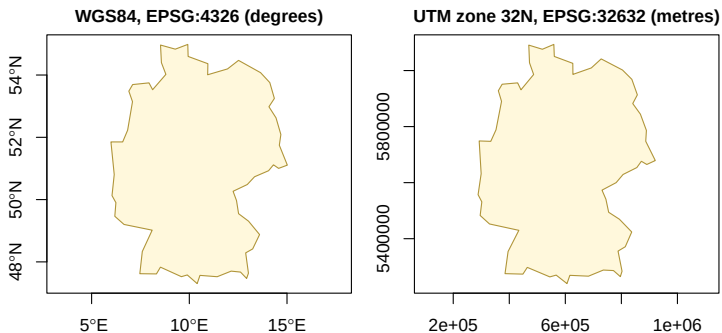
Measure in a projected CRS. But “projected” is not the whole question, and the second half of the question is what catches people. Hold that thought for twenty minutes.

UTM, the workhorse

- Peel the earth into 60 zones, each 6 degrees of longitude wide. Each zone gets its own central meridian and is projected separately.
- Within a zone, distortion in area and distance is under about 0.1 percent, and shapes stay right.
- Zone numbers run 1 to 60 eastward from 180 degrees West. Northern and southern hemispheres are separate codes.

The trick is refusing to project the whole world at once. Sixty small honest maps beat one large dishonest one. The cost is that two objects in different zones cannot be compared until you move them into the same one.

The same country, two coordinate systems



Same country, same data, different numbers on the axes and a visibly different shape. Neither is wrong. They answer different questions.

How a CRS is written down

```
st_crs(4326)$input           # what you asked for
```

```
## [1] "EPSG:4326"
```

```
st_crs(32632)$units         # what you will be measuring in
```

```
## [1] "m"
```

- An **EPSG code** is a short integer naming a CRS in a public registry. 4326 is WGS84 longitude and latitude.
- Older code writes a **proj4 string** instead, spelling the same thing out longhand. You will meet both. Look codes up at <https://epsg.io>.

Name projections by EPSG code, never by nickname

“Albers” is ambiguous, there are dozens. EPSG:5070 is not. A reader can look it up, and so can you in six months.

Real data is not always tidy

This is a shapefile from an actual published replication package.

```
eur <- st_read("../data/europe_modern.shp", quiet = TRUE)
st_crs(eur)$epsg                                # is there an EPSG code?
```

```
## [1] NA
```

```
substr(st_crs(eur)$proj4string, 1, 46)          # but there IS a definition
```

```
## [1] "+proj=eqdc +lat_0=30 +lon_0=10 +lat_1=43 +lat_"
```

- R knows the projection. It is an equidistant conic centred on Europe. There is simply no registry code for it.
- **So NA does not mean broken.** It means somebody built a custom projection, which is legitimate and common.
- What would be a genuine problem is a file with no definition at all. Then you have numbers and no idea what they mean.

A harder question than it looks

- Draw a closed loop on a flat map. It cuts the plane into two pieces, one finite and one stretching away forever. **Inside** is obviously the finite one. No convention required.
- Now draw the same loop on a globe. It still cuts the surface into two pieces, but on a sphere **both pieces are finite**.

The ambiguity

A ring drawn around France describes France just as well as it describes *the entire rest of the earth*. Nothing in the geometry distinguishes them. The data does not carry the answer.

This is not a puzzle about software. It is a fact about spheres.

So the software needs a convention

- A polygon is stored as an **ordered list of corner points**. Software walks them in that order to trace the boundary.
- Walking that order, you are going either clockwise or counterclockwise. That direction is called the **winding order**. It is nothing more exotic than which way round the list is written.

The rule

Walk the boundary in stored order. **The interior is on your left.**

Counterclockwise means you enclosed the small patch. Clockwise means you enclosed everything else.

On a flat map the winding order is mostly bookkeeping. On a sphere it is the *only* thing recording which of the two regions you meant.

The same four corners, written two ways

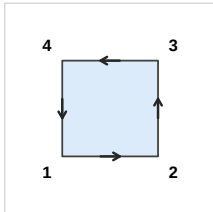
`## counterclockwise : 1,233,205 km2`

`## clockwise : 508,832,868 km2`

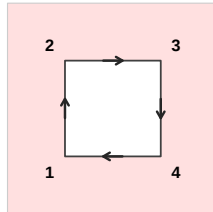
`## the two sum to : 510,066,073 km2`

`## earth's surface : 510,064,472 km2`

counterclockwise: the small patch



clockwise: everything else



Same four corners both times. The numbers are the stored vertex order and the arrows are the direction you walk it. Walking that order, the interior is on your left.

Why you have probably never hit this

```
## counterclockwise :      1,233,205 km2
```

```
## clockwise          :      1,233,205 km2
```

- `sf` asks `s2` for `oriented = FALSE`, which means “ignore the stored order and assume they meant the smaller region.” For countries and counties, that guess is always right.
- **The cost of the guess.** Under that default you cannot represent a region larger than half the earth. Ask for the ocean rather than the island and you will get the island.

Where the “Loop is not valid” errors come from

Planar GIS never had to care about ring order, so a great deal of published shapefile data is wound inconsistently. That is why `st_make_valid()` appears in almost every modern `sf` script, including the one that built these slides.

So why project at all? (1 of 2)

If spherical maths handles distance and area correctly, why project? Four reasons, in order of how often they will bind on you.

1. **Rasters are planar by construction.** You cannot tile a sphere with equal cells, by the same theorem that makes flat maps lie. Every gridded dataset you will touch, elevation, land use, night lights, climate, *is already projected*. From Week 4 on you are in a projected world whether you chose it or not.
2. **The statistics assume a plane.** Kernel weights, spatial lag matrices, kriging, and the Conley standard errors of Week 9 are all written in planar coordinates with Euclidean distance. You could rederive them on a manifold. Nobody has, and the software does not.

So why project at all? (2 of 2)

3. **Speed and robustness.** Spatial indexes are built on rectangles, and planar geometric predicates have decades of exact-arithmetic work behind them. At a few million features this stops being an aesthetic preference and starts being the difference between a script that finishes and one that does not.
4. **The data arrives that way.** Cadastral records and state plane systems are *legally defined* in projected feet. That is not a representation you chose. It is what the record is, and your paper has to speak its language.

Reason one is the one that will actually get you. Nobody decides to work in a projection. They open a raster.

The honest rule

It is not “always project”

That advice is a decade out of date and this deck has already shown you why. Measure on the sphere when you can. Project when you need constructive geometry, rasters, standard spatial statistics, or speed.

- Either way, the obligation is the same, and it is the only thing I actually want you to carry out of this section.
- **Know which regime you are in, and know what your units are.**

Notice what just happened. The rule everyone repeats turned out to be wrong, the reason is a genuine fact about spheres, and we settled it by running four lines of code rather than by asking who sounded most confident.

The three questions, every time you load anything

Ask in this order

1. `st_crs(x)` — is there a CRS at all?
2. Is it geographic or projected?
3. `st_crs(x)$units` — **what am I measuring in?**

Question three is the one almost nobody asks, and it is the one that is about to cost us in fifteen minutes. *Notice that none of this is hard. It is three function calls. The difficulty is entirely in remembering to make them before you trust a number.*

Verification, performed

The setup

We have two files. African country borders, and a table of cities with longitude and latitude columns.

```
africa <- st_read("data/africa_scale.shp")  
cities <- read_csv("data/africa_cities.csv")
```

- Nothing here is exotic. This is the shape of the first ten minutes of almost any spatial project.
- You will do exactly this yourself next week.

First question. What is it?

```
class(africa)
```

```
## [1] "sf" "data.frame"
```

```
st_crs(africa)$epsg
```

```
## [1] 4326
```

- An sf object *is* a data frame. One row per country, plus a geometry column.
- EPSG:4326 is longitude and latitude in **degrees**. Not metres. Remember that.

st_crs() is the second thing you run on anything you load. The first is class().

Now a real task

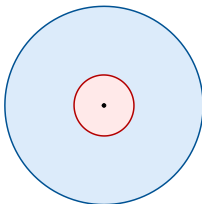
You are studying market towns. You want everything within **10 kilometres** of Fairfax. So you project to Virginia State Plane, which is the CRS every Virginia GIS office hands you, and you buffer.

```
fx_sp <- st_transform(fairfax, 2283)      # Virginia State Plane North  
buf   <- st_buffer(fx_sp, dist = 10000)   # 10 km. Yes?
```

- The CRS is projected, not geographic. So we did the thing everyone tells you to do.
- No error. No warning. An `sf` object comes back looking perfectly ordinary.

What we actually built

Intended 10 km (blue) versus what the code produced (red)



```
## radius you asked for : 10000 m
```

```
## radius you received  : 3047 m
```

```
## ratio                  : 0.3047
```

Why

```
st_crs(2283)$units
```

```
## [1] "us-ft"
```

Feet, not metres

EPSG:2283 is measured in **US survey feet**. So `dist = 10000` bought ten thousand **feet**, which is 3,047 metres. Not ten kilometres. A third of what you asked for.

- Every downstream number rests on that buffer. Every count of what fell inside it. Every treatment indicator.
- **And nothing complained.** The code ran. The object exists. A table built from it will look completely normal.

The diagnostic

So how would you have caught it? One line, before you trust anything.

```
st_crs(fx_sp)$units      # what am I measuring in?
```

```
## [1] "us-ft"
```

The habit

Before any `st_buffer`, `st_distance`, or `st_area`, ask what the units are. Not “is it projected”, which is the question everybody knows to ask. **What are the units.**

That is the difference between a rule you memorised and a rule you understand.

A second one. Count the rows

We want cities in a set of countries we care about.

```
keep <- tibble(admin = c("Kenya", "Uganda", "Tanzania"))  
  
nrow(cities)  
joined <- st_join(cities, africa) %>% inner_join(keep, by = "admin")  
nrow(joined)
```

- Perfectly reasonable code. A model would write it without hesitation, and so would most of us.
- The question is not whether it runs. It runs.

What the join did

```
## rows before : 2541
```

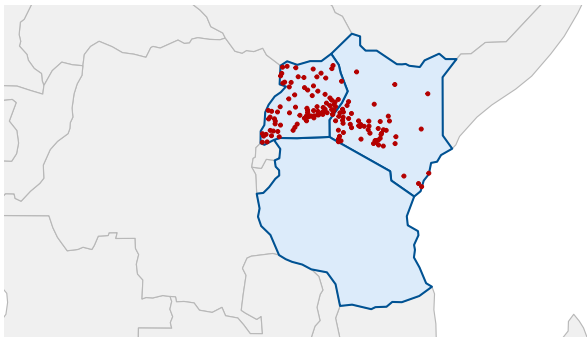
```
## rows after  : 144
```

```
## dropped      : 2397 (94%)
```

- Ninety-four percent of the rows are gone, which is roughly what you would expect. We asked for three countries out of fifty-one.
- Nothing errored. Nothing warned. The number is plausible, and plausible is where most people stop looking.

So we are finished. Unless we draw it.

Draw it



We asked for three countries and shaded all three. **Count the ones with cities in them.**

The join matched on a string

The shapefile calls it **United Republic of Tanzania**

`inner_join` compared "Tanzania" against that and found nothing. Two of the three names matched, one did not, and no part of R thought it worth mentioning. The 94 percent was right. The three countries were not.

- **I wrote this slide, and I did not catch it until I plotted the result.**

That is not a confession, it is the method. The row count looked reasonable, so it survived.

- **Count rows on both sides of every join, and then look at what you kept.** The count alone would never have found this.

What you just watched

Five habits, all of them cheap

1. Check the CRS of everything you load, and check its **units**.
2. Count rows before and after every join.
3. Plot early. A map that looks wrong is wrong.
4. Compute one number by hand and reconcile it.
5. Keep units attached as long as the software will let you.

None of that is advanced. All of it is the difference between a result and a published mistake. In arc two this list becomes the checklist you run against AI generated code.

Break

Ten minutes.

How the course runs

Three arcs

1. **Bare hands**, Weeks 1–5. You write the code yourself, because Week 6 tests whether you can.
2. **Tools on**, Weeks 6–9. The work shifts to specifying, verifying, and extending what the model produces.
3. **Text as data**, Weeks 10–12. Concepts first, so you can code intelligently with the AI.

Weeks 13 and 14 are your paper presentations.

Why the fundamentals come first

- You could not have caught either error tonight without knowing what a CRS is and what a join does.
- That is the entire argument for Weeks 1 to 5, and you have now seen it rather than been told it.

Nothing stops you using a model in Weeks 1–5. The lab exam at the start of Week 6 is closed book and closed model, and it tests exactly this material. That is what makes the bare weeks worth your time. *The constraint is an incentive, not a rule.*

Schedule, first half

Week	Date	Topic
1	Aug 24	Introduction and the verification frame
2	Aug 31	Vector data, distance, and IV's
—	Sep 7	No class, Labor Day
3	Sep 14	Buffers and regression discontinuity
4	Sep 21	Rasters and D-in-D
5	Sep 28	Making data, georeferencing and digitizing in QGIS
6	Oct 5	Lab exam , then LLM's as data constructors
—	Oct 12	No class, Fall Break

Schedule, second half

Week	Date	Topic
7	Oct 19	Network access and market access
8	Oct 26	Climate data and spatial interpolation
9	Nov 2	Cities, persistence, and standard errors in space
10	Nov 9	Text as data I, from corpus to matrix
11	Nov 16	Text as data II, embeddings
12	Nov 23	Text as data III, LLM's as measurement
13	Nov 30	Student presentations
14	Dec 7	Student presentations

Final paper due by midnight on Monday, December 14.

Assessment

- Research paper: **50%**

The heavy weights sit on the paper and the live presentation, the two things a model cannot do for you.

- Presentation: **30%**

- Lab exam, Week 6: **10%**

- Two exercises: **10%**

- Participation in discussion folds into the presentation grade.

- Near the end of the semester, a short oral spot check on the paper. Ten minutes walking me through your own code, answering one or two “what happens if” questions. Like tonight, but you are driving.

The exercises, build and audit

Each exercise has two halves

Build. Accomplish a set of spatial tasks in R, AI permitted and expected. Submit executable code plus a short note on what you asked the tools and what you checked.

Audit. You receive AI generated spatial code containing planted errors. Your grade is for finding them.

- Exercise 1 distributed Week 6, due Week 7. Exercise 2 distributed Week 8, due Week 9.
- **You have now done an audit.** Twice, in the last half hour. That is the whole exercise, with the planted errors being mine rather than a model's.

What gets planted

All real, all silent, all verified against the version of `sf` you will install

- A buffer of 10000 in a CRS whose units are feet. You saw this one.
- A join that quietly drops a fifth of the rows. You saw this one too.
- `st_set_crs()` where `st_transform()` was needed. The label moves and the coordinates do not.
- A categorical raster resampled bilinearly, producing land use class 2.7.

None of these throw an error. Finding them is the skill, and it is also the fun version of the skill.

The AI policy

- Use is **permitted and expected** on all coursework, both exercises included.
- Every submission carries a short **AI use note**. What you asked, what you checked.
- You sign the result. You own every number and every line.
- Bans on inputs are unenforceable and would only train the honest to lose. This course polices outputs and accountability instead.

One exception. The Week 6 lab exam is closed book and closed model. It is the only place in the course where the tools are off, and it is the only place they can be enforced off.

Which tools, and what they cost you

From Week 6 we work with **agentic coding tools**, meaning tools that read your files, write and run code, and iterate. Not a chat window you paste into. The difference matters, because the skill is directing and checking something that *acts*. I will demonstrate using Claude Code.

You are not required to buy anything

Mason gives every student **PatriotAI** free, which reaches Claude and ChatGPT models in a data-protected environment, and Copilot Chat comes with your Mason 365 account. Every graded piece of work here can be done with those. The cost is more copying and pasting, not a worse grade.

- Commercial agentic tools run about twenty dollars a month and are more pleasant. No grade depends on one.
- **If access is a difficulty, come and tell me in the next two weeks.** Not in October. That is a real offer and I would rather hear it early.

Your paper

The paper is half your grade

- Test a specific hypothesis with one or more tools from the course.
- About 15–20 pages, written in $\text{\LaTeX}$, due December 14.
- The best predictor of a good paper is a good question married to data nobody else has.

Novel data is what AI cheapens least

Anyone can prompt a model into running a regression on downloaded data. Georeferencing a map nobody has digitized, and knowing the judgment calls you made doing it, is a contribution the tools cannot commoditize.

Start hunting now

- Week 5 teaches you to make data from archival maps. Arrive with a candidate source.
- Good hunting grounds. Old atlases, colonial surveys, city plans, cadastral maps, thematic maps buried in old books and government reports.
- Ask of any source: what did somebody record about space that nobody has turned into data?

Students who leave the paper topic until after the presentations begin have, historically, a bad time. The course is sequenced so the tools arrive as your paper needs them, which only works if the paper exists.

Before next week

Logistics

- Mondays 7:20 to 10:00 pm, RSCH 201, Fairfax.
- Office hours by appointment, Buchanan Hall D159. Email njohnsoL@gmu.edu and we will find a time.
- Course page on Canvas. Syllabus, readings, exercise materials all live there.
- Read the assigned papers before the week they appear. The papers are the course.

Setup, and it is not optional

Do this before Sunday, not Monday at 7 pm

1. Install R and RStudio, current versions.
2. `install.packages(c("tidyverse", "sf", "terra"))`
3. Download `econ895_setup_test.R` from Canvas and run it.
4. It prints fourteen checks. If any says FAIL, it also tells you the fix. If the fix does not work, email me the console output.

On macOS and Linux, `sf` needs GDAL, GEOS, and PROJ underneath. That is where installs fail, and it is why the script checks them by name.

Before next week

- Run the setup script. Seriously.
- Read [Dittmar \(2011\)](#). It is the Week 2 paper and we will build its instrument in class.
- If R is new to you, work through the introductory videos linked on the syllabus. Budget real time.

Next week we do vector data, distance, and instrumental variables, and we compute distance from Mainz for real. Tonight was the argument. From here it is the work.

Tonight in one slide

1. The course teaches code so you can direct and audit the machine. Verification is the skill.
2. You sign the result. Every number, every line, whether you typed it or not.
3. You watched two silent errors tonight. Neither threw a warning. Both were caught by one cheap habit.
4. Three arcs, and the Week 6 lab exam is the hinge.
5. The paper is half your grade, and novel data is your edge. Start hunting.

References I

- Dell, M. (2010). The persistent effects of peru's mining mita. *Econometrica*, 78(6):1863–1903.
- Denny, M. J. and Spirling, A. (2018). Text preprocessing for unsupervised learning: Why it matters, when it misleads, and what to do about it. *Political Analysis*, 26(2):168–189.
- Dittmar, J. E. (2011). Information technology and economic change: the impact of the printing press. *The Quarterly Journal of Economics*, 126(3):1133–1172.
- Donaldson, D. and Hornbeck, R. (2016). Railroads and american economic growth: A “market access” approach. *The Quarterly Journal of Economics*, 131(2):799–858.
- Donaldson, D. and Storeygard, A. (2016). The view from above: Applications of satellite data in economics. *Journal of Economic Perspectives*, 30(4):171–98.
- Gentzkow, M., Kelly, B., and Taddy, M. (2019). Text as data. *Journal of Economic Literature*, 57(3):535–74.
- Michalopoulos, S. and Papaioannou, E. (2018). Spatial patterns of development: A meso approach. *Annual Review of Economics*, 10:383–410.
- Nunn, N. and Qian, N. (2011). The potato's contribution to population and urbanization: Evidence from a historical experiment. *The Quarterly Journal of Economics*, 126(2):593–650.